

Working with Text Data

Prof. Giuseppe Serra

Università degli Studi di Udine

Text Classification: Sentiment Analysis of Movie Reviews

I've been impressed with Chavez's stance against globalisation for sometime now, but it wasn't until I saw the film at the Amsterdam documentary international film festival that I realize what he has really achieved. This film tells the story of coup/conspiracy by Venezuela's elite, the oil companies and oil loving corrupt western governments, to remove democratically elected president Chavez, and return Venezuela back to a brutal dictatorship. This film is must for anyone who believes in freedom and justice, and is also a lesson to the rest of world ! I commend the people of Venezuela for taking matter into their own hands, and saving their country from the likes of Halliburton and the Bush regime.



Classifier



Classification
(Positive/Negative)

Preprocessing

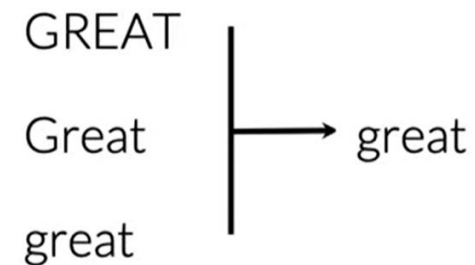
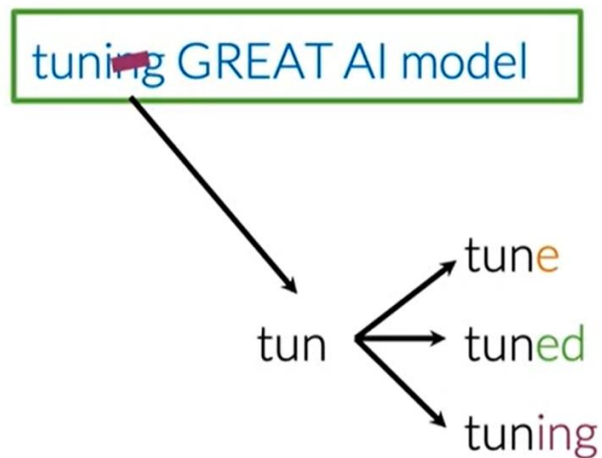
Preprocessing

Lower-casing: entire email is converted in lower case (HoME->home)

Word Stemming: words are reduced to their stemmed form.

- Example: "discount", "discounts", "discounted" and "discounting" are replaced with "discount"
- If this is done by using a rule-based heuristic, like dropping common suffixes ("ed", "ing" etc)

For instance:



Lemmatization

Lemmatization: is the process of grouping together the inflected forms of a word into a single item(lemma or dictionary form):

1. The word "better" has "good" as its lemma. This link is missed by stemming, as it requires a dictionary look-up.
2. The word "walk" is the base form for the word "walking", and hence this is matched in both stemming and lemmatisation.
3. The word "meeting" can be either the base form of a noun or a verb ("to meet") depending on the context; e.g., "in our last meeting" or "We are meeting again tomorrow". Unlike stemming, lemmatisation attempts to select the correct lemma depending on the context.

Lemmatization is computationally more difficult and expensive compared to stemming and it could have little impact on the performance.

Stop Words and Punctuation

Stop words usually refers to the most common words in a language like to", "I", "has", "the", "be", "or", etc.

In most cases, stop words add little semantic value to a sentence

Removing stop words helps decrease the size of representation without effect to scores.

NLTK (Natural Language Toolkit) in python has a list of stopwords stored in 16 different languages.

Punctuation can be also removed

We should think carefully, because they are could have semantic value in specific contexts.

Stop words	Punctuation
and	,
is	.
are	:
at	!
has	"
for	'
a	

Regular Expression

Customer normalization based on the domain

1. Removal of non-words: tabs, newlines etc are replace with single char.
2. Stripping HTML: HTML tags are removed.

....

Usually we use regular expression:

- great tutorial <https://developers.google.com/edu/python/regular-expressions>

I have been impressed with Chavez's stance against globalisation for sometime now, but it wasn't until I saw the film at the Amsterdam documentary international film festival that I realize what he has really achieved. This film tells the story of coup/conspiracy by Venezuela's elite, the oil companies and oil loving corrupt western governments, to remove democratically elected president Chavez, and return Venezuela back to a brutal dictatorship. This film is must for anyone who believes in freedom and justice, and is also a lesson to the rest of world ! I commend the people of Venezuela for taking matter into their own hands, and saving their country from the likes of Halliburton and the Bush regime.



Preprocessing Text



I have been impress with chavez stanc against globalis for sometim now but it wasn't until i saw the film at the amsterdam documentari intern film festiv that i realiz what he has reall achieved this film tell the stori of coup/conspiraci by venezuela elite the oil compani and oil love corrupt western governments to remov democrat elect presid chavez and return venezuela back to a brutal dictatorship. this film is must for anyon who believ in freedom and justice and is also a lesson to the rest of world! i commend the peopl of venezuela for take matter into their own hands and save their countri from the like of halliburton and the bush regime

Text Classification using Bag of Word Approach

Bag of Word Approach

Build a vocabulary list:

- We can choose the most frequently occurring words of our training set.
- In particular, we choose all the words which occur at least 100 times (in our exercise we use 5 times)
- The resulting list consists of 1899 words

```
1 aa
2 ab
3 been
...
85 oil
...
916 impress
...
1898 regime
1899 anyon
```

Bag of Word Approach

Mapping each word in the preprocessed email into a list of word indices that contains the index of the word in the vocabulary list:

Review

I have been impress with chavez stanc against globalis for sometim now but it wasn't until i saw the film at the amsterdam documentari intern film festiv that i realiz what he has reall achieved this film tell the stori of coup/conspiraci by venezuela elite the oil compani and oil love corrupt western governments to remov democrat elect presid chavez and return venezuela back to a brutal dictatorship. this film is must for anyon who believ in freedom and justice and is also a lesson to the rest of world! i commend the peopl of venezuela for take matter into their own hands and save their countri from the like of halliburton and the bush regime

Vocabulary list

1 aa
2 ab
3 been
...
85 oil
...
916 impress
...
1898 regime
1899 anyon

Word indices for the Review

22 44 3 916 1077 883 ...
370 1699 790 1822 1831
883 431 1171 ... 794
1002 1893 85 592 1676
238 162 85 688 945 1663
1120 ... 1062 1699 375
1162 479 1893 1510 799
1182 1237 85 1895
1440 ... 1547 181 1699
1758 1896 688 ... 1676
992 961 ... 1477 71 530
1699 1898

Bag of Word Approach

Feature Extraction from the email. Each email is converted in a vector in $x \in R^n$, where $n = \# \text{words in the vocabulary list (in our case 1899)}$

Specifically, $x_i = N$ when the i -th word appears in the email N times.

Word indices for the Email

22	44	3	916	1077	883	...
370	1699	790	1822	1831		
883	431	1171	...	794		
1002	1893	85	592	1676		
238	162	85	688	945	1663	
1120	...	1062	1699	375		
1162	479	1893	1510	799		
1182	1237	85	1895			
1440	...	1547	181	1699		
1758	1896	688	...	1676		
992	961	...	1477	71	530	
1699	1898					



$$x = \begin{bmatrix} 0 \\ \vdots \\ 2 \\ 0 \\ \vdots \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \in R^{1899}$$

Where, for example,

$$\begin{aligned} x_{85} &= 2 \\ &\vdots \\ x_{916} &= 1 \\ &\vdots \end{aligned}$$

Bag-of-words (EXAMPLE)

Training Set:

- The sun is shining
- The weather is sweet
- The sun is shining, the weather is sweet, and one and one is two

Vocabulary

- {'and': 0, 'is': 1, 'one': 2, 'shining': 3, 'sun': 4, 'sweet': 5, 'the': 6, 'two': 7, 'weather': 8}

Bag of Word representation

- [0 1 0 1 1 0 1 0 0]
- [0 1 0 0 0 1 1 0 1]
- [2 3 2 1 1 1 2 1 1]

The word 'is' occurs in all three sentences.

TF - IDF

Higher when w occurs many times within a small number of documents

Lower when the term occurs fewer times in a document, or occurs in many documents

It can be defined:

$$tfidf(w, d) = tf * \left(\ln \left(\frac{N + 1}{N_w + 1} \right) + 1 \right)$$

N = number of documents in the training set

N_w = number of documents in training set that w appears

tf = Term frequency (number of times that w appears in a document d)

In Scikit-learn the TF-IDF procedure include the L2-normalization

$$v_{norm} = \frac{v}{\|v\|_2} = \frac{v}{\sqrt{v_1^2 + v_2^2 + \dots + v_n^2}}$$

TF-IDF (EXAMPLE)

Training Set:

- The sun is shining
- The weather is sweet
- The sun is shining, the weather is sweet, and one and one is two

Vocabulary

- {'and': 0, 'is': 1, 'one': 2, 'shining': 3, 'sun': 4, 'sweet': 5, 'the': 6, 'two': 7, 'weather': 8}

TF-IDF

- [0. 0.43 0. 0.56 0.56 0. 0.43 0. 0.]
- [0. 0.43 0. 0. 0. 0.56 0.43 0. 0.56]
- [0.5 0.45 0.5 0.19 0.19 0.19 0.3 0.25 0.19]

In the 3rd document: 'is' (less discriminative) - 'and': 0 (more discr.)

TF-IDF (EXAMPLE)

Training Set:

- The sun is shining
- The weather is sweet
- The sun is shining, the weather is sweet, and one and one is two

$$tfidf(w, d) = tf * \left(\ln \left(\frac{N+1}{N_w+1} \right) + 1 \right)$$

$$idf("is", d3) = \ln \left(\frac{3+1}{3+1} \right) = \ln(1) = 0$$

$$tfidf("is", d3) = 3 * (0 + 1) = 3$$

Repeating these calculations for all terms in the 3rd document we have:

$$v = [3.39, 3.0, 3.39, 1.29, 1.29, 1.29, 2.0, 1.69, 1.29]$$

After L2-normalization $tfidf("is", d3) = 0.45$

Bag of Word Approach

Limitation:

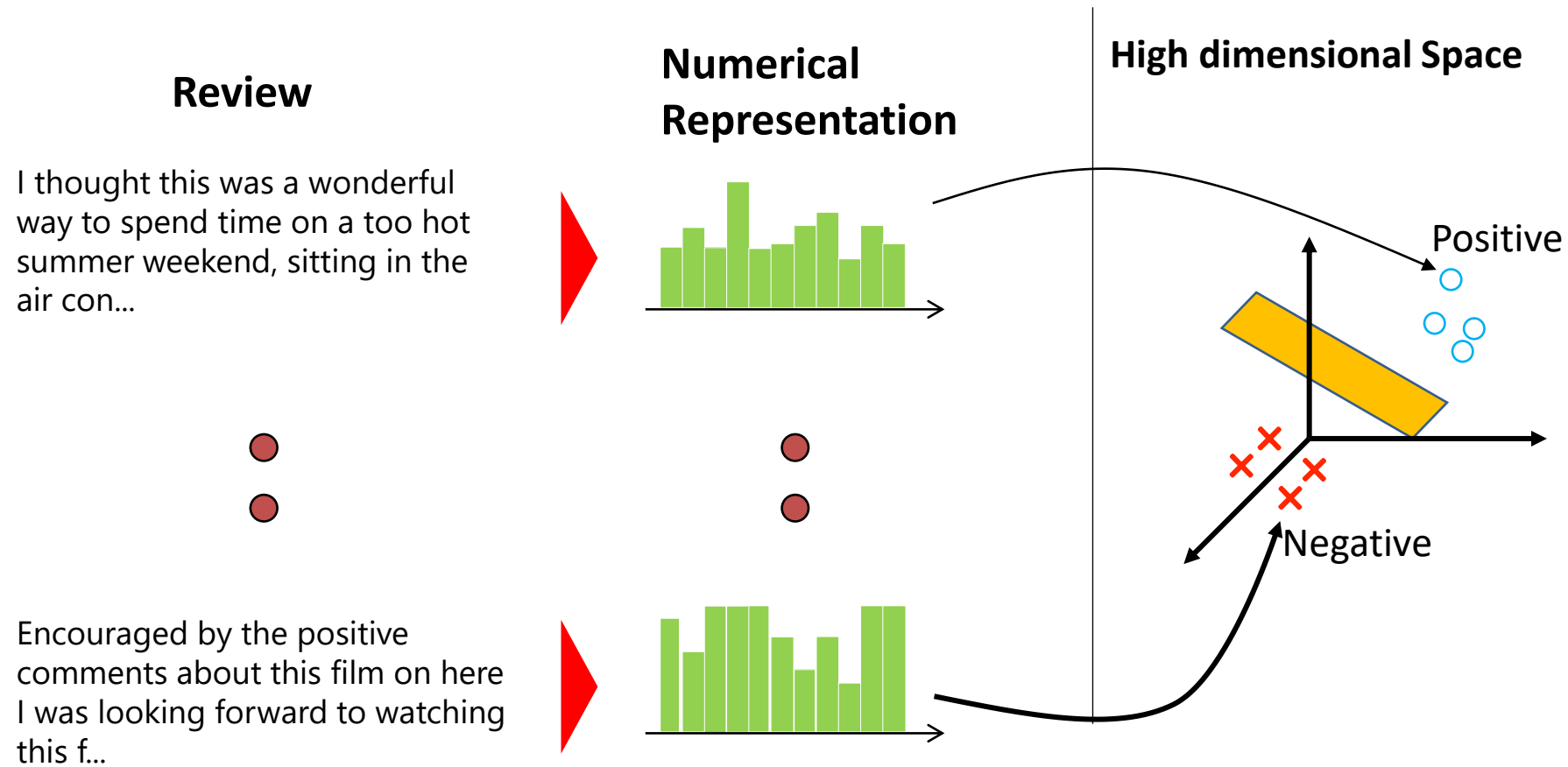
- "Mary is smarter than John"
- "John is smarter than Mary"
- These two sentences have the same bag-of-word representation, but they have different meaning!!

Capturing the context

- No considering just single tokens
- Count pairs or triples of tokens (bigrams and trigrams)
- So, in the vocabulary we could have bigrams like "very good" or "not worth"

Word Representations

Text Classification with BoW

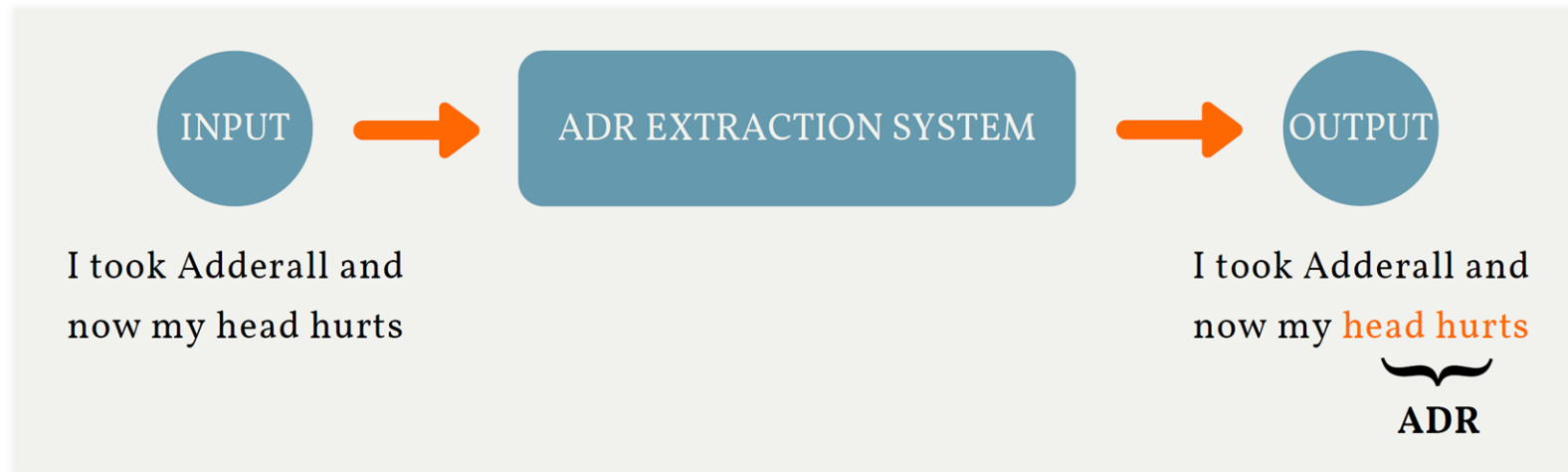


Bag of Word vs Word Representations

Bag of word technique is a simple and effective technique, but the representation is a global representation of sentence or a documents.

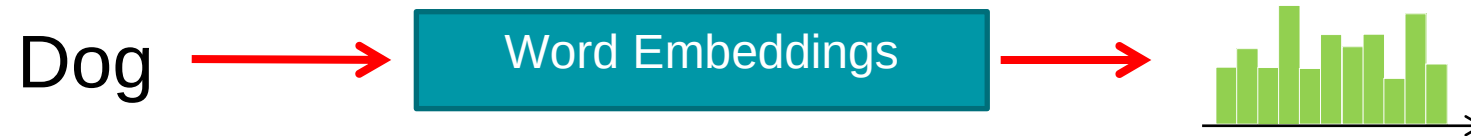
It could be useful in several NLP tasks such as Text Classification or Topic Discovery

But it is not suitable for task when we want to predict words or extract spans of text such as keyphrase extraction, Named entity Extraction



Bag of Word vs Word Representations

Word representation techniques represent words as feature vectors.



Combined with Deep Learning these techniques are state of the art in NLP in almost all the tasks.

In the following slides we will see basic and more advance techniques.

Basic Word Representations - Integers

The simplest way to represent words as numbers is for a given vocabulary to assign a unique integer to each word.

Word	Number
a	1
able	2
about	3
...	...
hand	615
...	...
happy	621
...	...
zebra	1000

Basic Word Representations - Integers

This numbering scheme is simple,

One of the problems is that the order of the words,

For instance, alphabetical order doesn't make much sense from a semantic perspective.

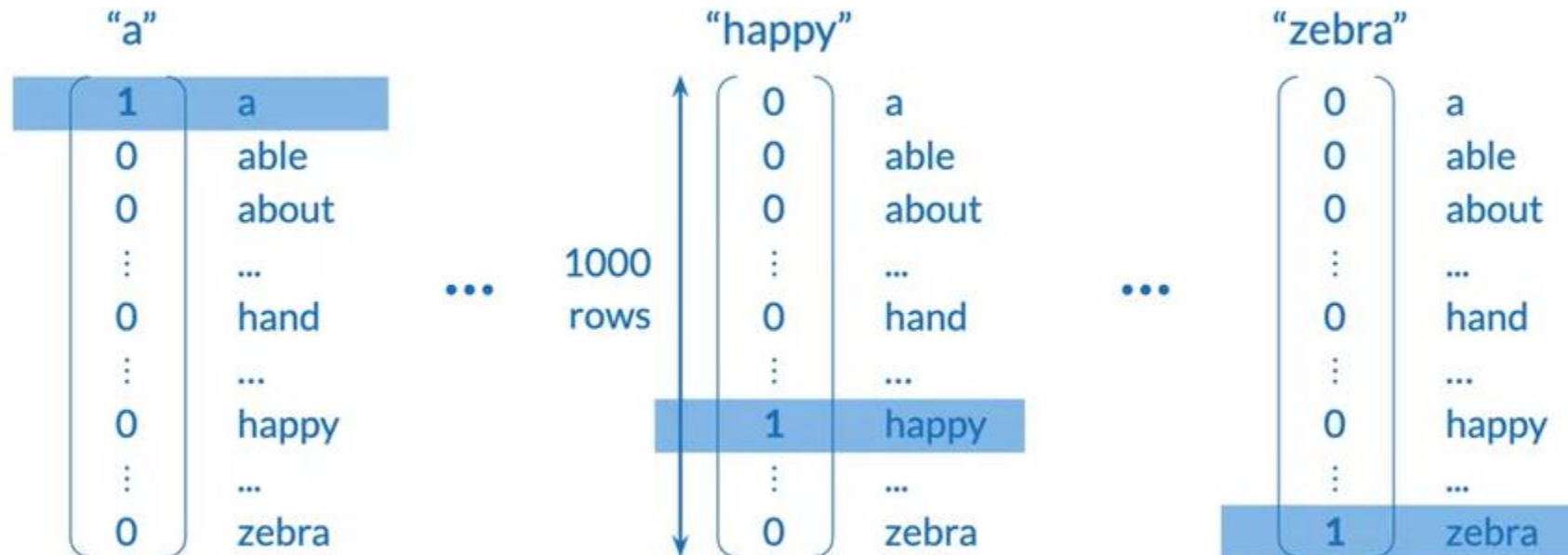
- + Simple

- Ordering: little semantic sense

hand	<	happy	<	zebra
615	?!<	621	?!<	1000

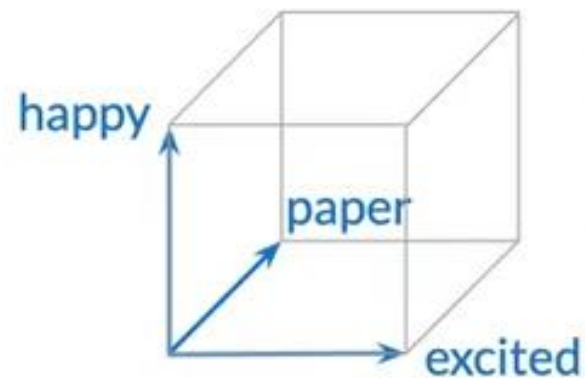
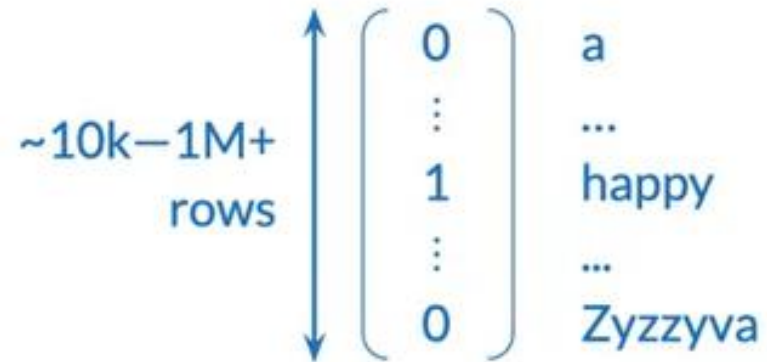
Basic Word Representations – One-hot Vectors

One-hot Vectors represent the words using a column vector where each element corresponds to a word of the vocabulary.



Basic Word Representations – One-hot Vectors

- + Simple
- + No implied ordering
- Huge vectors
- No embedded meaning



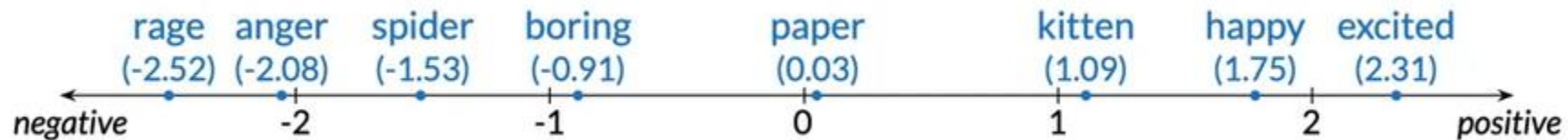
$$\begin{aligned} d(\text{paper}, \text{excited}) \\ &= d(\text{paper}, \text{happy}) \\ &= d(\text{excited}, \text{happy}) \end{aligned}$$

Basic Word Representations – Word Embeddings

Word embeddings are vectors that are carrying meaning.

Consider a horizontal number line like this. Words on the left are considered negative in some way and words on the right are considered positive in some.

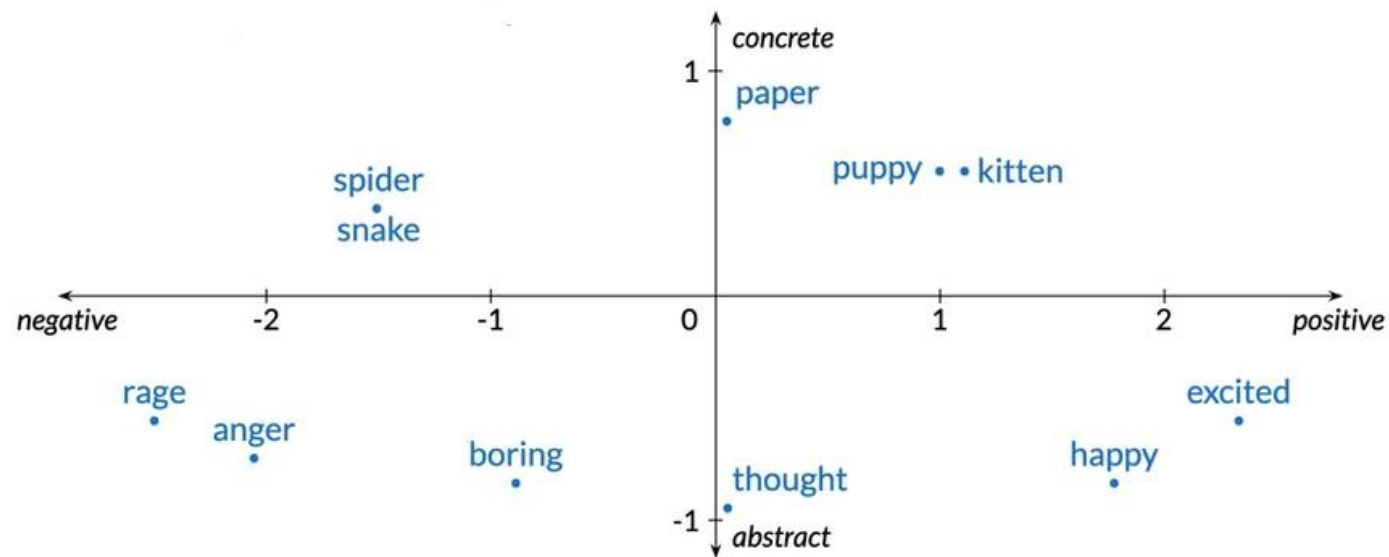
Now you can say that's happy and excited are more similar to each other compared to with the word paper



Basic Word Representations – Word Embeddings

You can extend this by adding a vertical number line, words that are higher on this line are more concrete physical objects. Whereas words lower on this line are more abstract ideas.

What you've done here is to represent the vocabulary of words with a small vector of length 2.

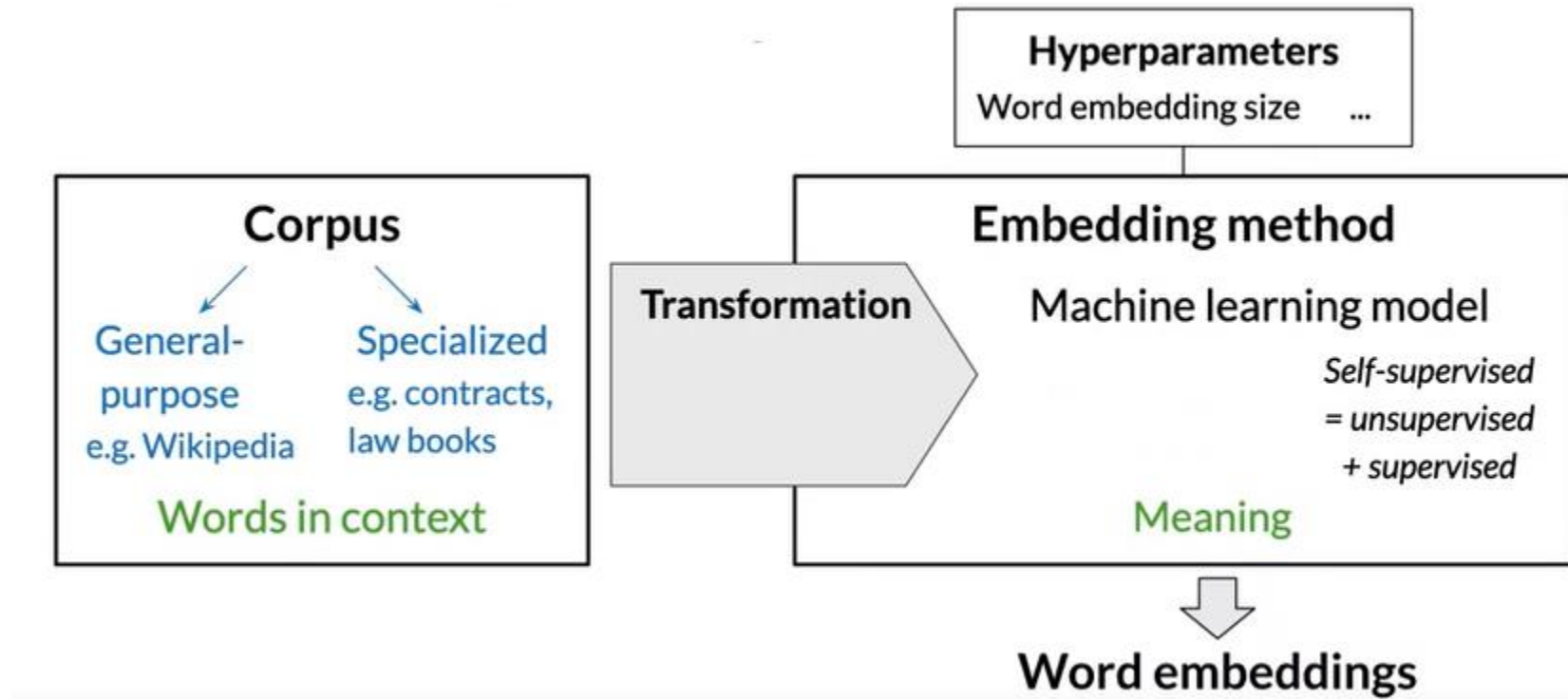


Basic Word Representations – Word Embeddings

- + Low dimension
- + Embed meaning
 - o e.g. semantic distance

forest $\approx$ tree forest $\neq$ ticket

Basic Word Representations – Word Embedding Process



More info: https://radimrehurek.com/gensim/auto_examples/tutorials/run_word2vec.html#sphx-glr-auto-examples-tutorials-run-word2vec-py

